

What is data replication?

Data replication is the process of keeping copies of the same data across multiple locations so your system stays available, consistent, and fast regardless of where users access it. When data changes on a primary database, those changes propagate to one or more replicas so every copy stays aligned.

It's worth separating this from backup. Backups are point-in-time snapshots you restore from after something goes wrong. Replication is an ongoing process that keeps copies current — often within milliseconds of a change — so you're still running while something is going wrong.

As your data footprint grows and users spread across regions, that distinction matters more, not less.

How does data replication work?

Data replication works by synchronizing changes from a primary data source to one or more replica destinations. A primary node (also called a source) handles write operations and logs every change, then those changes propagate to replica nodes, which receive and apply the updates.

The process typically involves three stages. First, an initial state is established through a full snapshot or baseline copy of the primary database. Second, ongoing changes are captured — either by reading transaction logs, monitoring timestamps on updated records, or using database triggers. Third, those changes are applied to each replica, with timing determined by whether the architecture is synchronous or asynchronous.

The most common technique for capturing and streaming changes from a primary database is change data capture (CDC). CDC reads the database's transaction log to detect inserts, updates, and deletes with minimal impact on the primary system, preserving the original order of transactions without requiring schema changes on the source.

Even if the link between a primary and a replica breaks mid-operation, a partial resynchronization replays only missed commands from the replication log. If those commands have already rolled out of the primary's replication backlog, a full resynchronization occurs instead: the primary generates and sends a complete snapshot to the replica.

What are the benefits of data replication?

- Improved reliability & disaster recovery
- Increased app performance
- More efficient engineering teams

Types of data replication

Those availability and performance benefits come from different replication strategies, each with different timing and scope trade-offs.

The main differences are when data is copied and how much of the dataset each replica receives.

Synchronous replication

In synchronous replication, data is written to both the primary and replica simultaneously. The primary waits for confirmation that the replica has received and applied the write before acknowledging the transaction as complete. This keeps the two aligned, but the round-trip to the replica adds latency to every write operation. Synchronous replication works best for in-region deployments where network latency between nodes is low.

Asynchronous replication

In asynchronous replication, the primary acknowledges the write immediately and propagates changes to replicas afterward. This eliminates the latency penalty on write operations but introduces replication lag, a window where replicas may be slightly behind the primary. Asynchronous replication is the practical choice for long-distance or multi-region deployments where synchronous round-trips would add unacceptable latency. The trade-off: a primary failure during the lag window could mean a small amount of data hasn't reached the replica yet.

Transactional replication

Transactional replication is a replication topology that copies data changes in real time, preserving the exact order of changes made on the primary. It begins with a full snapshot of the primary database, then streams subsequent changes as they occur.

A snapshot of the primary is shared with the replica.

The primary later sends changes gathered after the snapshot.

Because of its incremental nature, transactional replication is well suited for scenarios where every change must be reflected

everywhere. It is not typically used as a standalone backup strategy.

Snapshot replication

Snapshot replication copies the state of the data at a specific point in time and moves that copy to the replica. It captures nothing after the snapshot is taken, so it doesn't stay current on its own. Snapshot replication is useful as a baseline for initializing new replicas or for low-volatility datasets where periodic refreshes are acceptable — similar to reverting to an earlier saved version of a document.

Merge replication

Merge replication starts with an initial snapshot, then allows each node to make independent changes that are later reconciled into a unified dataset. This is useful in distributed environments where nodes need to operate independently (field teams working offline who sync when they reconnect, for example). Conflict resolution logic determines which changes take precedence when two nodes have modified the same record independently.

Key-based replication

Key-based incremental replication uses a replication key — typically a timestamp or incrementing ID column — to identify only the records that have changed since the last replication cycle. It is fast and places minimal load on the source system. The primary limitation is that it doesn't replicate deleted records, since deletions remove the row that would otherwise be detected by the replication key.

Understanding full vs. partial data replication

Full database replication copies the entire primary database to every replica, mirroring all existing, new, and updated data across the system. Every replica holds the complete dataset, but the approach requires substantial network bandwidth and storage at every location.

Partial replication mirrors only selected data — typically the most recently updated records or data relevant to a specific region or workload. A headquarters database might replicate all records globally while regional offices replicate only what their users access locally. This reduces bandwidth requirements and keeps replicas smaller, at the cost of more complex routing logic when requests touch data that isn't local.

What are the common challenges of data replication?

Data replication adds operational complexity even as it improves availability and performance.

Infrastructure cost

Running replicas means additional hardware or cloud instances alongside your primary, and every replica that needs to stay current adds compute, storage, and network costs. The right number depends on your availability requirements and budget.

Replication lag

In any asynchronous setup, applications that read from a replica immediately after a write may see stale results. Addressing this, through read-your-own-writes routing, synchronous replication on critical paths, or eventual consistency patterns at the application layer, requires deliberate architectural decisions upfront.

Conflict resolution

Conflict resolution becomes necessary whenever multiple nodes can accept writes. Two nodes may independently update the same record before synchronizing, and the system needs a deterministic way to resolve those conflicts: last-write-wins policies, application-defined rules, or conflict-free replicated data types (CRDTs) that merge concurrent changes without discarding either update.

Bandwidth consumption

Replicating large datasets or high-throughput write workloads across regions generates substantial network traffic. Log-based CDC reduces data in flight by sending only changed rows rather than full table copies, at the cost of more complex setup and monitoring.

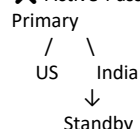
Active-Active Geo Distribution

What is it?

Active-Active = Every region/server can both READ and WRITE.

Instead of having one primary:

☒ Active-Passive



Active-Active looks like:

☒ Active-Active

US Region

↑ ↑
Reads Writes
↑ ↑
India Region
↑ ↑
Reads Writes
Every node is **active** and serves real traffic.

How does it work?

Suppose a user in India writes data:

User (India)
↓
India Node
↓
Local Write ☒
↓
Sync with other regions
A user in the US can also write directly to the US node:

User (US)
↓
US Node
↓
Local Write ☒
↓
Sync with other regions
The nodes continuously synchronize their changes.

Why is it useful?

1. Lower latency

Users write to their **local region**.

India user → India server
US user → US server
So we don't need to wait for a cross-region write.

2. No single primary

There isn't one server responsible for all writes.

3. Better failure handling

If one region goes down:

India ✗
US ☒
Europe ☒
Other regions can continue accepting traffic.
When India comes back, it can **resynchronize**.

What about conflicting writes?

Imagine:

US node:
balance = 100
India node:
balance = 100
Both modify it at nearly the same time.
This creates a **conflict**.

Active-Active systems need a way to resolve these conflicts.

The source mentions **CRDTs (Conflict-free Replicated Data Types)** for automatically resolving many conflicts.

For some data types, such as Strings, **Last-Write-Wins (LWW)** can be used